Software Requirements Specification

Hotel Management System

WinForms Desktop Edition

Field	Value
Document Version	1.0
Date	2026-05-08
Prepared by	Abdurahman Ibrahem
Standard	IEEE Std 830-1998

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the **Hotel Management System (HMS)** — a single-user, Windows desktop application that supports front-desk, housekeeping, restaurant, and management activities of a small-to-mid-sized hotel.

The intended audience includes:

- **The development team** building, testing, and maintaining the system.
- **The course instructor / academic reviewers** evaluating compliance with IEEE 830-1998.
- **Hotel staff and managers** who will operate the system and provide acceptance feedback.
- **QA engineers** who will derive test cases from the requirements stated herein.

This document is the authoritative reference for what the system *shall* do. Behaviors not covered here are out of scope unless added through a controlled change to this document.

1.2 Scope

The product, hereafter referred to as **HMS**, is a desktop Windows Forms application implemented on the .NET 9 platform. It allows authorized hotel personnel to:

- Authenticate against role-based credentials (Manager, Staff).
- Manage rooms, room conditions, and rates.
- Manage guest profiles, including VIP designation and stay history.
- Create, confirm, cancel, check-in, and check-out reservations.
- Operate an in-house restaurant: maintain a menu, place orders against active stays, and progress orders through a kitchen workflow.
- Generate, view, settle (mark paid), and refund itemized invoices that combine room and restaurant charges with tax.
- View management reports such as occupancy, revenue by room type, top-selling menu items, average stay duration, and repeat-guest percentage.

Out of scope (v1.0):

- Multi-tenant / multi-property deployment.
- Online booking, third-party channel manager, or public-facing web portal.
- Real payment gateway integration; payment status is recorded but no money moves.
- Networked / multi-user concurrent access; the system is single-machine, single-user per launch.

- Persistent storage in a relational database (data is in-memory and reseeds on launch — see §2.4).

The benefits of HMS are: reduced front-desk paperwork, consistent invoice arithmetic, fewer double-bookings, and a unified view of room occupancy and revenue for managers.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
HMS	Hotel Management System — the product specified by this document.
SRS	Software Requirements Specification (this document).
GUI	Graphical User Interface.
WinForms	Microsoft Windows Forms — the UI toolkit on which HMS is built.
CRUD	Create, Read, Update, Delete.
VIP	A guest flagged as receiving priority service, typically with a high stay count.
Stay	The active occupation of a room by a guest, between check-in and check-out.
Reservation	A booking made for future occupancy of a specific room by a specific guest.
Invoice line	A single chargeable item on an invoice (room-night or restaurant item).
Tax rate	A fixed 10% applied to invoice subtotals (see FR-INV-3).
Order workflow	The state machine `Placed` → `Preparing` → `Ready` → `Served`, plus a terminal `Cancelled` state reachable from `Placed` or `Preparing`.

1.4 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. Microsoft .NET 9 SDK Documentation — https://learn.microsoft.com/dotnet/.
3. Microsoft Windows Forms Documentation — https://learn.microsoft.com/dotnet/desktop/winforms/.
4. Project source repository: `Windows/` (working tree of this SRS).

1.5 Overview

Section 2 provides a high-level overview of the product, its users, and its constraints. Section 3 enumerates the detailed functional and non-functional requirements, organized by feature, with unique identifiers (FR-*, NFR-*) suitable for traceability and acceptance testing. Section 4 contains supporting matrices and appendices.

2. Overall Description

2.1 Product Perspective

HMS is a **self-contained desktop product**. It is not a component of, nor a successor to, any other system. It is composed of three architectural tiers running in a single .NET process:

```
+-----------------------------------------------------------+
|  Presentation tier (WinForms)                             |
|    LoginForm, MainForm (tabbed dashboard), CheckoutForm   |
+-----------------------------------------------------------+
|  Service tier (domain logic, no UI dependencies)          |
|    AuthService     RoomService      BookingService        |
|    RestaurantService  InvoiceService  ReportService       |
+-----------------------------------------------------------+
|  Data tier                                                |
|    DataStore (in-memory BindingList<T> collections)       |
|    SeedData (bootstraps demo rooms, users, menu, stays)   |
+-----------------------------------------------------------+
```

System-context view:

```
      +-------------+
      |   Operator  |
      |  (Manager / |
      |    Staff)   |
      +------+------+
             |
             v keyboard / mouse
      +------+----------------+
      |        HMS            |---> Display (screen)
      |  (single .NET process)|---> Printer (future, out of scope v1)
      +-----------------------+
```

The product **has no external interfaces** to other software systems in v1.0 (no network calls, no databases, no payment gateways, no email).

2.2 Product Functions

At the highest level, HMS supports the following functions. Each is elaborated as a numbered functional requirement in §3.2.

1. **Authentication & session management —** username/password login, role-based menu visibility.
2. **Room management —** CRUD, rate setting, condition tracking (Clean / Needs Cleaning / Out of Service), maintenance log.
3. **Guest management —** register guests, mark VIPs, track stay count.
4. **Reservation lifecycle —** create, confirm, cancel, check-in, check-out.

5. **Stay management —** track active stays, accumulated room and restaurant charges.
6. **Restaurant operations —** menu CRUD, take orders against an active stay, advance orders through kitchen states.
7. **Invoicing —** generate invoices on checkout (or on-demand), itemize room nights and restaurant lines, apply 10% tax, record payment status and method.
8. **Reporting & dashboard —** occupancy rate, revenue breakdowns, top menu items, average stay duration, repeat-guest percentage, today's revenue.

2.3 User Characteristics

Two user classes are recognized:

Class	Education / Experience	Frequency of Use	Privileges
Staff (front desk)	Computer-literate; minimal technical training expected. Trained in HMS basics in <1 hour.	Heavy daily use (reservations, check-in/out, restaurant orders).	Operate reservations, stays, restaurant orders, take payments. **Cannot** add/remove rooms or menu items, **cannot** access management reports beyond basic dashboard counters.
Manager	Computer-literate; familiar with basic business reports.	Daily monitoring + ad-hoc admin tasks.	All Staff capabilities **plus** room CRUD, menu CRUD, full reports, refunds.

The system is designed so that a Staff user can complete a typical check-in workflow with at most six clicks and no command-line knowledge.

2.4 Constraints

The following constraints are imposed on the product and its development:

1. **CON-1 (Platform):** The product shall run on Windows 10 (1809+) and Windows 11 with .NET 9 Desktop Runtime installed.
2. **CON-2 (Toolchain):** The product is developed in C# 12 / .NET 9 with WinForms and built via `dotnet build`.
3. **CON-3 (Storage):** Data resides in process memory only. There is no persistence layer in v1.0; on each launch the system is reseeded from `SeedData.cs`. (Persisted storage is identified as a future enhancement — see §2.6.)
4. **CON-4 (Single-user):** Only one operator at a time uses one running instance. Concurrent multi-user access on the same data is not supported.
5. **CON-5 (Locale & currency):** All monetary values are displayed in U.S. dollars ($) with two-decimal formatting. Dates use the operating system's short date format.

6. **CON-6 (Tax rate):** Sales tax is fixed at 10% (hard-coded in `Invoice.Tax`). A configurable tax rate is out of scope for v1.0.
7. **CON-7 (Security — academic context):** Passwords are stored in plaintext within seeded data because the product is an academic prototype. A production deployment would require hashing (BCrypt/Argon2), per NFR-SEC-1.
8. **CON-8 (Coding standards):** Source code follows the user's global rules for .NET / C# (Onion-style layering — Presentation/Services/Data; async/await for I/O when introduced; unit tests for core service logic).

2.5 Assumptions and Dependencies

- **ASM-1:** Operators have valid Windows user sessions and physical access to the host machine.
- **ASM-2:** The host machine has sufficient resources (≥ 2 GB RAM, ≥ 100 MB free disk) — see NFR-PRF-1.
- **ASM-3:** The system clock is correct; date-driven logic (nights billed, today's revenue) trusts `DateTime.Now`.
- **ASM-4:** The hotel does not require regulatory PCI-DSS compliance for v1.0 because no real card data is processed.
- **DEP-1:** Microsoft .NET 9 Desktop Runtime is installed on the target machine.
- **DEP-2:** The product depends on the Windows Forms component (`UseWindowsForms=true`) and therefore cannot run on Linux or macOS without changes.

2.6 Apportioning of Requirements

The following requirements are deferred beyond v1.0 and are **not** acceptance criteria for the current release:

- Persistent storage (SQLite or SQL Server via EF Core).
- Networked multi-user mode with optimistic concurrency.
- Payment gateway integration (Stripe, Square).
- Receipt printing and PDF export.
- Localization for non-English locales.
- Role-granular permissions beyond Staff/Manager.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces (UI)

The application exposes the following top-level windows:

ID	Window	Purpose
UI-1	**LoginForm**	Capture username and password; on successful login, close itself and launch MainForm. On failure, display an inline error and clear the password field.
UI-2	**MainForm**	A tabbed dashboard with the following primary tabs: Dashboard, Rooms, Reservations, Stays, Restaurant, Invoices, Reports. The visible set is filtered by role per FR-AUTH-3.
UI-3	**CheckoutForm**	Modal dialog to finalize a stay: shows itemized invoice preview, total with tax, payment method dropdown, and Mark Paid / Cancel buttons.

UI-wide rules:

- **UI-R1:** All monetary values shall be displayed using format `$#,##0.00`.
- **UI-R2:** All editable date fields shall use a `DateTimePicker` constrained to valid ranges (e.g., check-out > check-in).
- **UI-R3:** Destructive actions (cancel reservation, remove room, refund invoice) shall require an explicit confirmation dialog.
- **UI-R4:** Actions disallowed by current state shall be disabled (greyed out) rather than producing an error after-the-fact.
- **UI-R5:** Lists (rooms, reservations, stays, orders, invoices) shall update automatically when their underlying collections change (`BindingList<T>`).

3.1.2 Hardware Interfaces

The product requires only standard PC hardware: keyboard, mouse, and a display of at least 1280×720. No specialized hardware (card readers, key encoders, biometric scanners) is interfaced in v1.0.

3.1.3 Software Interfaces

- **SI-1:** Operating system — Windows 10 (1809+) or Windows 11.
- **SI-2:** Runtime — .NET 9 Desktop Runtime.
- **SI-3:** No external services, databases, message queues, or APIs are consumed.

3.1.4 Communications Interfaces

None. HMS performs no network communication in v1.0.

3.2 Functional Requirements

Each requirement is identified by a stable ID of the form `FR-<area>-<n>`. Areas: AUTH, ROOM, GUEST, RES, STAY, RST (restaurant), INV (invoice), RPT (reports), DSH (dashboard).

3.2.1 Authentication & Session

- **FR-AUTH-1:** The system shall authenticate users by case-insensitive username and case-sensitive password against the seeded user store.
- **FR-AUTH-2:** Upon successful authentication, the system shall set a current-user context accessible to all forms during the session.
- **FR-AUTH-3:** The system shall enforce role-based access:
  - **Staff** can: view rooms, manage reservations, perform check-in/out, take restaurant orders, settle invoices.
  - **Manager** can: do everything a Staff user can, plus add/edit/remove rooms, add/edit/remove menu items, refund invoices, view full reports.
- **FR-AUTH-4:** The system shall provide a Logout action that clears the current user and returns to the LoginForm.
- **FR-AUTH-5:** On three consecutive failed login attempts within one session, the LoginForm shall display a warning. (No lockout in v1.0; this is informational only.)

3.2.2 Room Management

- **FR-ROOM-1:** The system shall list all rooms with: number, type, rate, occupancy, condition, and computed display status.
- **FR-ROOM-2 (Manager only):** The system shall allow adding a new room. Room number must be unique; attempting to add a duplicate shall raise a validation error and reject the addition.
- **FR-ROOM-3 (Manager only):** The system shall allow editing a room's number, type, and rate. Uniqueness of room number shall be re-checked on update.
- **FR-ROOM-4 (Manager only):** The system shall allow removing a room **only if** it is not occupied. Occupied rooms shall fail removal with an explanatory error.
- **FR-ROOM-5:** The system shall support marking a room: Occupied, Vacant, Needs Cleaning, Clean, or Out of Service.
- **FR-ROOM-6:** When a room is marked Out of Service, the system shall require a maintenance reason, which is stored in the room's maintenance log.
- **FR-ROOM-7:** The system shall consider a room available **iff** it is not occupied **and** its condition is `Clean`. Out-of-service or dirty rooms shall not be selectable for new reservations or check-ins.
- **FR-ROOM-8:** On check-out, the system shall automatically transition the room to `NeedsCleaning`.

3.2.3 Guest Management

- **FR-GUEST-1:** The system shall allow creating a guest with name and contact information, optionally flagged as VIP.
- **FR-GUEST-2:** The system shall maintain a `StayCount` per guest, automatically incremented on each successful check-out (FR-RES-5).
- **FR-GUEST-3:** The system shall allow viewing a guest's history of past reservations and stays.

3.2.4 Reservation Lifecycle

- **FR-RES-1:** A reservation shall be created with a guest, an available room, a check-in date, and a check-out date strictly later than the check-in date.
- **FR-RES-2:** On creation, a reservation's status shall be `Confirmed`.
- **FR-RES-3:** `Confirmed` reservations may be cancelled, transitioning to `Cancelled`. A `Cancelled` reservation cannot be reactivated.
- **FR-RES-4:** `Confirmed` reservations may be checked-in, which:
  - sets the reservation status to `CheckedIn`,
  - marks the room occupied,
  - creates an `Active` stay record with the same guest and room and the actual check-in timestamp set to `now`,
  - sets the stay's `ExpectedCheckOut` to the reservation's check-out date.
- **FR-RES-5:** An `Active` stay may be checked-out, which:
  - records the actual check-out timestamp (`now`),
  - sets the stay status to `CheckedOut`,
  - computes room charges as `max(1, floor(actual − checkIn in days)) × rate`,
  - aggregates non-cancelled restaurant charges into the stay,
  - marks the room vacant and `NeedsCleaning`,
  - increments the guest's `StayCount`,
  - transitions the source reservation (if any) to `Completed`.
- **FR-RES-6:** Once a stay is `CheckedOut`, it is read-only and shall not appear in active-stay lists.

3.2.5 Restaurant Operations

- **FR-RST-1:** The system shall maintain a menu of items with name, price, category, availability flag, and description.
- **FR-RST-2 (Manager only):** Add, edit, and remove menu items.
- **FR-RST-3:** Any user may toggle a menu item's availability. Unavailable items are excluded from new orders but remain on existing orders.
- **FR-RST-4:** A restaurant order is associated with exactly one Active stay. Orders for non-Active stays shall not be creatable.

- **FR-RST-5:** Each order line has a menu item, a positive quantity, and optional notes. Line total = `quantity × menuItem.price`.
- **FR-RST-6:** A new order's status shall be `Placed`. The system shall expose an "advance" action that transitions the status `Placed` → `Preparing` → `Ready` → `Served`. No backwards transitions.
- **FR-RST-7:** An order may be cancelled **only** while in `Placed` or `Preparing`. Cancelled orders are excluded from billing aggregations.
- **FR-RST-8:** The system shall allow appending lines to an existing order while it is in `Placed` or `Preparing`. Adding lines shall recompute the parent stay's accumulated `RestaurantCharges` from non-cancelled orders.

3.2.6 Invoicing

- **FR-INV-1:** The system shall generate an invoice for a Stay containing:
  - one `RoomCharge` line per night of stay (`max(1, floor(end − start in days))`), each priced at the room's rate, dated by night,
  - one `RestaurantCharge` line per order line on every non-cancelled order belonging to the stay.
- **FR-INV-2:** Each invoice shall have a system-generated, monotonically increasing identifier of the form `INV-NNNN` starting at `INV-1001`.
- **FR-INV-3:** The system shall compute:
  - **Subtotal** = sum of line totals,
  - **Tax** = `round(Subtotal × 0.10, 2)`,
  - **Total** = `Subtotal + Tax`.
- **FR-INV-4:** A new invoice's `PaymentStatus` shall be `Pending`.
- **FR-INV-5:** The system shall provide a "Mark Paid" action that records `PaymentMethod ∈ {Cash, CreditCard, DebitCard, BankTransfer}` and a `PaymentDate = now`, transitioning the status to `Paid`.
- **FR-INV-6 (Manager only):** The system shall provide a "Refund" action that transitions a `Paid` invoice to `Refunded`.
- **FR-INV-7:** The system shall list **outstanding** invoices (status `Pending`) and report their summed total.
- **FR-INV-8:** All invoice line edits shall be prohibited after the invoice is created. Adjustments require a new invoice or refund.

3.2.7 Reporting & Dashboard

- **FR-RPT-1:** The system shall compute the live **occupancy rate** as `occupiedRooms / totalRooms × 100%`. If no rooms exist, occupancy is `0`.

- **FR-RPT-2:** The system shall compute **revenue by room type**, summing `RoomCharges` of all stays grouped by room type.
- **FR-RPT-3:** The system shall compute **restaurant revenue by category**, summing line totals across all orders grouped by menu category.
- **FR-RPT-4:** The system shall list the **top N menu items** (default N=5) by total quantity sold across all orders.
- **FR-RPT-5:** The system shall compute the **average stay duration** in days across all checked-out stays.
- **FR-RPT-6:** The system shall compute the **repeat guest percentage** as the fraction of guests whose `StayCount > 1`.
- **FR-RPT-7:** The system shall compute **today's served-order revenue** as the sum of totals on all orders with status `Served` and `CreatedAt.Date == today`.
- **FR-RPT-8:** The system shall compute **today's invoice revenue** as the sum of totals on invoices `Paid` today.
- **FR-DSH-1:** The MainForm dashboard shall present, at minimum: occupancy rate, today's revenue, count of active stays, count of pending invoices, and top 5 menu items.

3.3 Non-Functional Requirements

3.3.1 Performance

- **NFR-PRF-1:** The application shall launch (cold-start to LoginForm visible) within **3 seconds** on a 4-core 8 GB Windows 10/11 machine.
- **NFR-PRF-2:** Common interactive operations (open a tab, list rooms, create a reservation, advance an order, generate an invoice) shall complete within **300 ms** at seed-data scale (≤ 50 rooms, ≤ 500 stays, ≤ 5000 order lines).
- **NFR-PRF-3:** Memory footprint at idle shall not exceed **250 MB** of working set.

3.3.2 Reliability

- **NFR-REL-1:** A handled exception in any service operation shall not crash the application; the offending action shall be aborted and an error dialog shown.
- **NFR-REL-2:** Invalid state transitions (e.g., checking out a non-active stay, removing an occupied room) shall be rejected at the service layer with a descriptive `InvalidOperationException`.
- **NFR-REL-3:** Numeric calculations for invoices shall use `decimal` arithmetic (no `double`/`float`) to avoid binary-floating-point rounding errors.

3.3.3 Usability

- **NFR-USE-1:** A trained Staff user shall complete a check-in within **30 seconds** measured from selecting a reservation to confirming the stay.

- **NFR-USE-2:** All forms shall be navigable by keyboard (Tab order, Enter to submit primary action, Esc to cancel modal).
- **NFR-USE-3:** Validation errors shall be shown next to or referencing the offending field, not as opaque message boxes.
- **NFR-USE-4:** The product shall use a consistent visual theme (see `Theme/`) across all forms.

3.3.4 Maintainability

- **NFR-MNT-1:** The codebase shall maintain a strict separation between Presentation (`Forms/`), Service (`Services/`), Model (`Models/`), and Data (`Data/`) layers. UI code shall not reference `DataStore` directly except through service interfaces.
- **NFR-MNT-2:** Each service class shall have unit tests for its core logic (per global coding rules). v1.0 ships with placeholders; full coverage is a v1.1 deliverable.
- **NFR-MNT-3:** Public service methods shall use intention-revealing names; no method shall exceed 40 lines without justification.

3.3.5 Security

- **NFR-SEC-1:** Passwords **shall** be hashed using a modern adaptive algorithm (BCrypt or Argon2id) before any production deployment. Plaintext seed credentials are acceptable in the academic build only and shall be flagged in release notes.
- **NFR-SEC-2:** The application shall not log credentials, payment data, or guest contact information at any verbosity.
- **NFR-SEC-3:** Role checks shall be enforced **at the service layer**, not only by hiding UI controls. UI hiding is defense-in-depth, not the primary control.

3.3.6 Portability

- **NFR-PRT-1:** The product targets `net9.0-windows`. Cross-platform support is explicitly out of scope.
- **NFR-PRT-2:** The product shall not depend on machine-specific paths; configuration (when introduced) shall live next to the executable or in `%AppData%/HotelManagement`.

3.3.7 Availability

- **NFR-AVL-1:** As a desktop product, availability is a function of the host machine. The product shall not introduce background services or daemons that require maintenance.

3.4 System Features (Use Cases)

This section restates the principal flows as use cases so that test cases can be derived directly.

UC-1: Log In

Field	Description
Actor	Staff or Manager
Pre	Application launched; LoginForm visible.
Main flow	1. User enters username and password. 2. User clicks Login. 3. System validates credentials. 4. System opens MainForm with role-appropriate tabs.
Alt	3a. Credentials invalid → inline error displayed; password cleared; remain on LoginForm.
Post	`AuthService.CurrentUser` is set; MainForm is open.
Traces	FR-AUTH-1, FR-AUTH-2, FR-AUTH-3

UC-2: Create Reservation

Field	Description
Actor	Staff
Pre	Logged in; an available room exists; guest record exists or is created inline.
Main flow	1. User opens Reservations tab → New. 2. User selects guest, available room, check-in date, check-out date. 3. System validates that check-out > check-in and room is available. 4. System persists Reservation with status Confirmed.
Alt	3a. Date invalid → error message; reservation not created. 3b. Room becomes unavailable mid-flow → user re-selects.
Post	A new Confirmed reservation appears in the list.
Traces	FR-RES-1, FR-RES-2, FR-ROOM-7

UC-3: Check In

Field	Description
Actor	Staff
Pre	A `Confirmed` reservation exists for today or earlier; its room is `Available`.
Main flow	1. User selects the reservation → Check In. 2. System creates an Active stay, marks the room occupied, sets reservation status CheckedIn.
Post	Stay appears in Stays; room shows `Occupied`.
Traces	FR-RES-4, FR-ROOM-5

UC-4: Place Restaurant Order

Field	Description
Actor	Staff
Pre	An `Active` stay exists; at least one available menu item.
Main flow	1. User opens Restaurant tab → selects stay. 2. User adds menu items with quantities. 3. User clicks Place Order. 4. System creates RestaurantOrder with status Placed.
Alt	At any point user may Cancel the draft.
Post	Order visible on the stay's order list.
Traces	FR-RST-4, FR-RST-5, FR-RST-6

UC-5: Advance Order

Field	Description
Actor	Staff (kitchen view)
Main flow	1. User selects an order. 2. User clicks Advance. 3. System transitions status one step (`Placed` → `Preparing` → `Ready` → `Served`).
Alt	If status is `Served`, the Advance button is disabled.
Traces	FR-RST-6

UC-6: Check Out & Generate Invoice

Field	Description
Actor	Staff
Pre	`Active` stay selected.
Main flow	1. User clicks Checkout. 2. CheckoutForm opens with itemized preview. 3. User selects payment method. 4. User clicks Mark Paid. 5. System: closes stay, marks room `NeedsCleaning`, creates invoice, marks invoice `Paid` with the chosen method, increments guest `StayCount`.
Alt	4a. User clicks Cancel → stay remains `Active`; no invoice generated.
Post	Stay is `CheckedOut`; invoice in Invoices is `Paid`.
Traces	FR-RES-5, FR-INV-1, FR-INV-3, FR-INV-5, FR-ROOM-8

UC-7: Refund Invoice

Field	Description
Actor	Manager
Pre	A `Paid` invoice.
Main flow	1. Manager opens invoice → Refund. 2. System asks for confirmation. 3. System sets status `Refunded`.
Traces	FR-INV-6, NFR-SEC-3

UC-8: View Reports

Field	Description
Actor	Manager
Main flow	User opens Reports tab; system displays occupancy, revenue by room type, restaurant revenue by category, top menu items, average stay duration, repeat-guest percentage.
Traces	FR-RPT-1 … FR-RPT-8

3.5 Logical Database / Data Model

Although v1.0 has no relational store, the in-memory model defines the conceptual schema that a future persistence layer would realize.

3.5.1 Entity Overview

```
User (Username PK, Password, Role)

Room (Number PK, Type, Rate, IsOccupied, Condition, MaintenanceLog)

Guest (Id PK*, Name, Contact, IsVip, StayCount)

MenuItem (Id PK*, Name, Price, Category, IsAvailable, Description)

Reservation (Id PK*, GuestId FK→Guest, RoomNumber FK→Room,
             CheckInDate, CheckOutDate, Status)

Stay (Id PK*, GuestId FK→Guest, RoomNumber FK→Room,
      CheckInDate, ExpectedCheckOut, ActualCheckOut?,
      RoomCharges, RestaurantCharges, Status)

RestaurantOrder (Id PK*, StayId FK→Stay, Status, CreatedAt)
OrderLine (Id PK*, OrderId FK→RestaurantOrder, MenuItemId FK→MenuItem,
           Quantity, Notes)

Invoice (InvoiceNumber PK, StayId FK→Stay, GuestId FK→Guest,
```

```
        RoomNumber FK→Room, InvoiceDate, PaymentStatus,
        PaymentMethod?, PaymentDate?)
InvoiceLine (Id PK*, InvoiceId FK→Invoice, Description,
             Quantity, UnitPrice, Category)
```

* In v1.0 the in-memory store uses object references; surrogate keys are introduced when a database is added.

3.5.2 Key Constraints

- **DC-1:** `Room.Number` is unique.
- **DC-2:** `Reservation.CheckOutDate > Reservation.CheckInDate`.
- **DC-3:** `Stay.ActualCheckOut`, when set, satisfies `ActualCheckOut ≥ CheckInDate`.
- **DC-4:** `OrderLine.Quantity ≥ 1`.
- **DC-5:** `MenuItem.Price ≥ 0`.
- **DC-6:** `Invoice.PaymentMethod` is set **iff** `PaymentStatus ∈ {Paid, Refunded}`.
- **DC-7:** A `Stay` has at most one source `Reservation`.

3.5.3 State Machines

Reservation:

```
[*] -> Pending -> Confirmed -> CheckedIn -> Completed
              \-> Cancelled
```

Stay:

```
[*] -> Active -> CheckedOut
```

Order:

```
[*] -> Placed -> Preparing -> Ready -> Served
            \-> Cancelled
                Preparing -> Cancelled
```

Invoice:

```
[*] -> Pending -> Paid -> Refunded
```

3.6 Design Constraints

- **DSC-1:** The product shall conform to the user's global C# / .NET coding rules (Onion-style layering, async/await for I/O, unit tests for core logic).
- **DSC-2:** No business logic shall live in `Forms/*.Designer.cs` or in repositories; presentation code orchestrates services only.
- **DSC-3:** All cross-list updates shall use `BindingList<T>` so WinForms data-bound controls refresh automatically (NFR-USE).
- **DSC-4:** Currency arithmetic uses `decimal` (CON-3, NFR-REL-3).

4. Supporting Information

4.1 Traceability Matrix

Feature / Use Case	Functional Requirements	Non-Functional
Login (UC-1)	FR-AUTH-1..5	NFR-SEC-1, NFR-USE-2
Create Reservation (UC-2)	FR-RES-1, FR-RES-2, FR-ROOM-7, FR-GUEST-1	NFR-PRF-2, NFR-USE-3
Check In (UC-3)	FR-RES-4, FR-ROOM-5	NFR-PRF-2
Restaurant Order (UC-4, UC-5)	FR-RST-1..8	NFR-PRF-2
Checkout & Invoice (UC-6)	FR-RES-5, FR-INV-1..5, FR-ROOM-8, FR-GUEST-2	NFR-REL-3, NFR-USE-1
Refund (UC-7)	FR-INV-6, FR-AUTH-3	NFR-SEC-3
Reports (UC-8)	FR-RPT-1..8, FR-DSH-1	NFR-PRF-2
Room CRUD	FR-ROOM-1..8	NFR-MNT-1
Menu CRUD	FR-RST-1..3	NFR-MNT-1

4.2 Appendices

Appendix A — Seeded Credentials (Academic Build Only)

Username	Password	Role
`admin`	`admin123`	Manager
`staff`	`staff123`	Staff

> *These credentials exist solely to allow demonstration without registration. They must be removed and replaced with hashed credentials before any deployment outside of academic evaluation (NFR-SEC-1, CON-7).*

Appendix B — Worked Invoice Example

A guest stays 3 nights in Room 302 (`$249.99`/night) and orders one Beef Steak (`$32.99`) and two Cappuccinos (`$4.99` each).

```
Subtotal = 3 × 249.99 + 1 × 32.99 + 2 × 4.99
         = 749.97 + 32.99 + 9.98
         = 792.94
Tax      = round(792.94 × 0.10, 2) = 79.29
Total    = 792.94 + 79.29          = 872.23
```

This corresponds exactly to the `INV-1001` seeded record, validating FR-INV-1 and FR-INV-3.

Appendix C — Glossary of Services and Their Primary Responsibilities

Service	Responsibility
`AuthService`	Login/Logout; current user.
`RoomService`	Room CRUD; condition transitions; availability query.
`BookingService`	Reservation create/cancel; check-in/check-out; charge accumulation.
`RestaurantService`	Menu CRUD; order create/advance/cancel; line append; revenue today.
`InvoiceService`	Invoice generate; mark paid/refunded; revenue queries; outstanding list.
`ReportService`	Aggregations for the Reports tab and dashboard.

Appendix D — Change Log

Version	Date	Author	Notes
1.0	2026-05-08	Abdurahman Ibrahem	Initial IEEE-830 SRS for v1.0 academic release.

— End of Document —